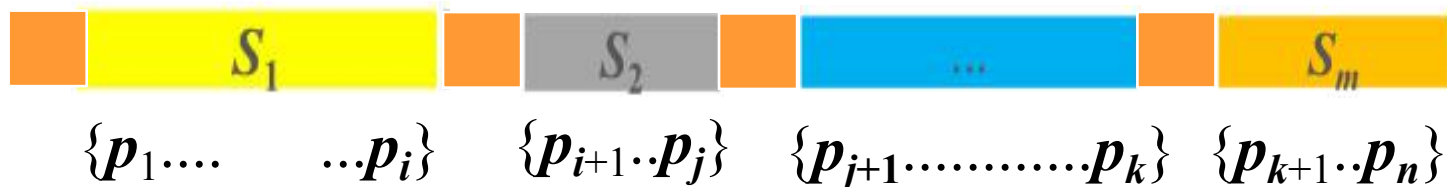


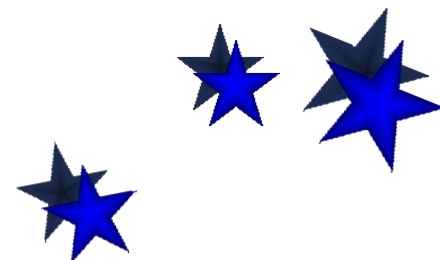
3.3 动态规划算法的典型应用

◆ 图像压缩存储：分段变位存储。



n 个像素点占用的总位数=

$$l[1] \times b[1] + l[2] \times b[2] + \dots + l[m] \times b[m] + \mathbf{11m}$$



3.3 动态规划算法的典型应用

➤ 子问题界定与计算顺序

子问题前边界为1，后边界为 i

对应像素序列为 $\langle p_1, p_2, \dots, p_i \rangle$

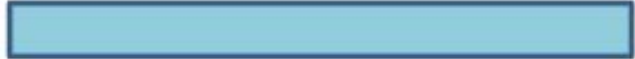
优化函数值 $S[i]$ 中存储子问题 i 的最优分段存储位数。

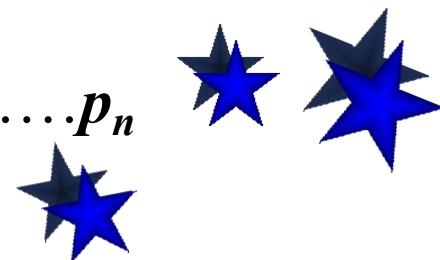
计算顺序

$i = 1$  $p_1 \dots p_1$

$i = 2$  $p_1 \dots p_2$

...

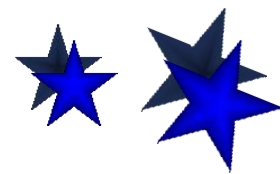
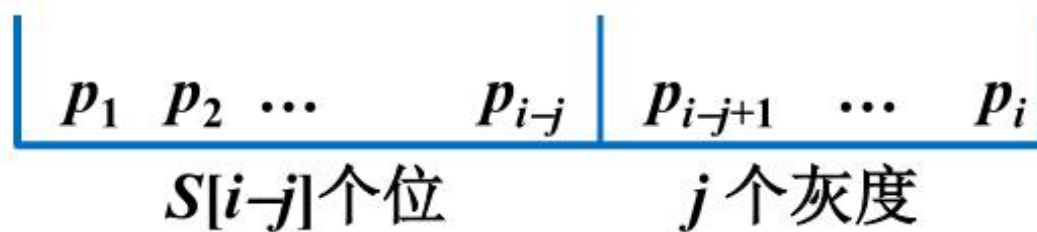
$i = n$  $p_1 \dots p_n$



3.3 动态规划算法的典型应用

➤ 状态转移方程

- 在对子问题 i 进行决策时，子问题0到子问题 $i-1$ 的结果已经存储到 $S[0]$ 、 $S[1]$ $S[i-1]$ 中。
- 问题 i 的最后一个分段 S_m 的长度为 j ($1 \leq j \leq \min(i, 256)$)
 - (1) 前 $m-1$ 个分段中共有 $i-j$ 个像素点，其最优解已经存储到 $S[i-j]$ 中，可直接使用。
 - (2) S_m 段中有 j 个像素点，每个像素点的位数由 j 个像素点的最大值决定。



3.3 动态规划算法的典型应用

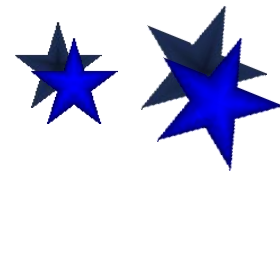
(2) 如何确定分段 S_m 占用的位数？

① S_m 长度=1时， S_m 中只含有 p_i ，计算其存储位数，记入 $b[i]$ 中。

② S_m 长度=2时， S_m 中含有 $p_{i-1}p_i$ ，计算 p_{i-1} 的存储位数并与 $b[i]$ 比较，将其中较大者记入 $b[i-1]$ 。

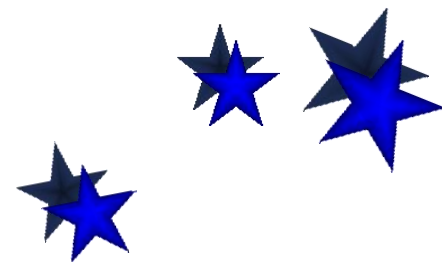
③ S_m 长度= j 时， S_m 中含有 $p_{i-j+1}\cdots p_{i-1}p_i$ ，计算 p_{i-j+1} 的存储位数并与 $b[i-j+2]$ 比较，将其中较大者记入 $b[i-j+1]$ 。

- 在对不同长度的 S_m 进行决策时，应事先计算该段包含像素的最大存储位数，并将其存储于 $b[i-j+1]$ 中。



3.3 动态规划算法的典型应用

- 数组***b***中的内容，不同阶段值是不同的，例如
 - (1) 第1阶段(p_1): 仅使用***b***[1]，存储 p_1 的位数。
 - (2) 第2阶段(p_1, p_2): 使用***b***[1]、***b***[2]，***b***[2]中存储 p_2 的位数，***b***[1]中存储 $\max\{p_1, p_2\}$ 。
 - (3) 第*i*阶段($p_1 \dots p_i$): 使用***b***[1].....***b***[*i*]，***b***[*i*]中存储 p_i 的位数，***b***[*j*]中存储 $\max\{p_{i-j+1} \dots p_i\}$ ，***b***[1]中存储 $\max\{p_1 \dots p_i\}$ 。



3.3 动态规划算法的典型应用

- 求解子问题 i 的过程:

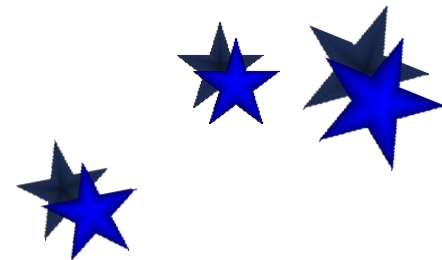
	S_m 长度	S_m 的位数	子问题 $i-j$	结果
$S[i]=$ min	1	$b[i]+11$	$S[i-1]$	$S[i-1]+b[i]+11$
	2	$2*b[i-1]+11$	$S[i-2]$	$S[i-2]+2*b[i]+11$
				
	j	$j*b[i-j+1]+11$	$S[i-j]$	$S[i-j]+j*b[i-j+1]+11$
				
	i	$n*b[1]+11$	$S[0]$	$S[0]+n*b[1]+11$

$$S[i]=\min\{S[i-j]+j*b[i-j+1]\}+11$$
$$1\leq j\leq\min(i,256)$$



3.3 动态规划算法的典型应用

- 实例: $P = \langle 10, 12, 15, 255, 1, 2 \rangle$
- 使用的数据结构:
 - (1) 数组 S : 存储每阶段求出的最优解, 例如 $S[i]$ 存储 $p_1 \cdots p_i$ 的最优解。
 - (2) 数组 b : 例如 $b[i-j+1]$ 中存储每阶段计算出的 $p_j \cdots p_i$ 每个像素点存储需要的位数。
 - (3) 数组 l : 标记函数 (追踪解), 用于记录分段的位置。
例如: 若 $l[i]$ 中记录的值为 x , 则说明最后一个段应包括 x 个像素点, 即 $p_{i-x+1} \cdots p_i$ 。



3.3 动态规划算法的典型应用

$P = \langle 10, 12, 15, 255, 1, 2 \rangle$

(1) 第一阶段 (10)

$$b[1]=4$$

$$S[1]=\min\{S[0]+1*4\}+11=15$$

$$l[1]=1$$

S	15	19				
l	1	2				

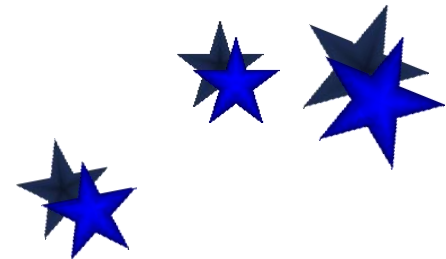
(2) 第一阶段 (10, 12)

$$j=1 \quad b[2]=4 \quad S[1]+1*4=19$$

$$j=2 \quad b[1]=4 \quad S[0]+2*4=8$$

$$S[2]=\min\{19+8\}+11=19$$

$$l[2]=2$$



3.3 动态规划算法的典型应用

(3) 第三阶段 (10, 12, 15)

$$j=1 \quad b[3]=4 \quad S[2]+1*4=23$$

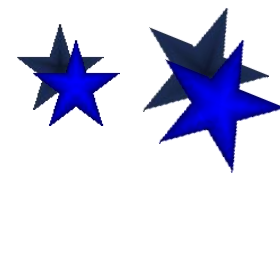
$$j=2 \quad b[2]=4 \quad S[1]+2*4=23$$

$$j=3 \quad b[1]=4 \quad S[0]+3*4=12$$

$$S[3]=\min\{23, 23, 12\}+11=23$$

$$l[3]=3$$

S	15	19	23			
l	1	2	3			



3.3 动态规划算法的典型应用

(4) 第四阶段 (10, 12, 15, 255)

$$j=1 \quad b[4]=8 \quad S[3]+1*8=31$$

$$j=2 \quad b[3]=8 \quad S[2]+2*8=35$$

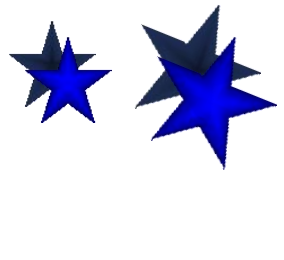
$$j=3 \quad b[2]=8 \quad S[1]+3*8=39$$

$$j=4 \quad b[1]=8 \quad S[0]+4*8=32$$

$$S[4]=\min\{31,35,39,32\}+11=42$$

$$l[4]=1$$

<i>S</i>	15	19	23	42		
<i>l</i>	1	2	3	1		



3.3 动态规划算法的典型应用

(5) 第五阶段 (10, 12, 15, 255, 1)

$$j=1 \quad b[5]=1 \quad S[4]+1*1=43$$

$$j=2 \quad b[4]=8 \quad S[3]+2*8=39$$

$$j=3 \quad b[3]=8 \quad S[2]+3*8=43$$

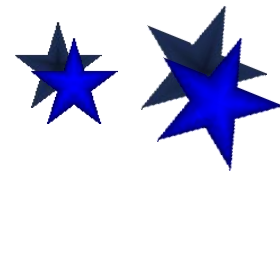
$$j=4 \quad b[2]=8 \quad S[1]+4*8=47$$

$$j=5 \quad b[1]=8 \quad S[0]+5*8=40$$

$$S[5]=\min\{43,39,43,47,40\}+11=50$$

$$l[5]=1$$

<i>S</i>	15	19	23	42	50	
<i>l</i>	1	2	3	1	2	



3.3 动态规划算法的典型应用

(6) 第六阶段 (10, 12, 15, 255, 1, 2)

$$j=1 \quad b[6]=2 \quad S[5]+1*2=52$$

$$j=2 \quad b[5]=2 \quad S[4]+2*2=46$$

$$j=3 \quad b[4]=8 \quad S[3]+3*8=47$$

$$j=4 \quad b[3]=8 \quad S[2]+4*8=51$$

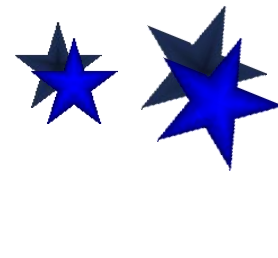
$$j=5 \quad b[2]=8 \quad S[1]+5*8=55$$

$$j=6 \quad b[1]=8 \quad S[0]+6*8=48$$

$$S[6]=\min\{52,46,47,51,55,48\}+11=57$$

$$l[6]=1$$

S	15	19	23	42	50	57
l	1	2	3	1	2	2



3.3 动态规划算法的典型应用

- 解的追踪

S	15	19	23	42	50	57
l	1	2	3	1	2	2

(1) 最少占用的位数: $S[6]=57$

(2) 分段, 从后向前的分段:

$l[6]=2$, 表示 $p_1—p_6$, 最后一个分段有2个像素
 $6-2=4$,

$l[4]=1$, 表示从 $p_1—p_4$, 最后一个分段有1个像素
 $4-1=3$

$l[3]=3$, 表示从 $p_1—p_3$, 最后一个分段有3个像素
 $3-3=0$, 结束



伪码

Compress (P, n)

1. $Lmax \leftarrow 256$; $header \leftarrow 11$; $S[0] \leftarrow 0$

2. for $i \leftarrow 1$ to n do

3. $b[i] \leftarrow \text{length}(P[i])$

4. $bmax \leftarrow b[i]$

5. $S[i] \leftarrow S[i-1] + bmax$

6. $l[i] \leftarrow 1$

7. for $j \leftarrow 2$ to $\min\{i, Lmax\}$ do

8. if $bmax < b[i-j+1]$

9. then $bmax \leftarrow b[i-j+1]$

10. if $S[i] > S[i-j] + j * bmax$

11. then $S[i] \leftarrow S[i-j] + j * bmax$

12. $l[i] \leftarrow j$

13. $S[i] \leftarrow S[i] + header$

子问题后
边界 i

最后
段长 j

找到更
好分段



追踪解

算法 Traceback (n, l)

输入：数组 l

输出：数组 C

1. $j \leftarrow 1$ // j 为正在追踪的段数
2. while $n \neq 0$ do
3. $C[j] \leftarrow l[n]$ 
4. $n \leftarrow n - l[n]$
5. $j \leftarrow j + 1$

$C[j]$: 从后向前追踪的第 j 段的长度

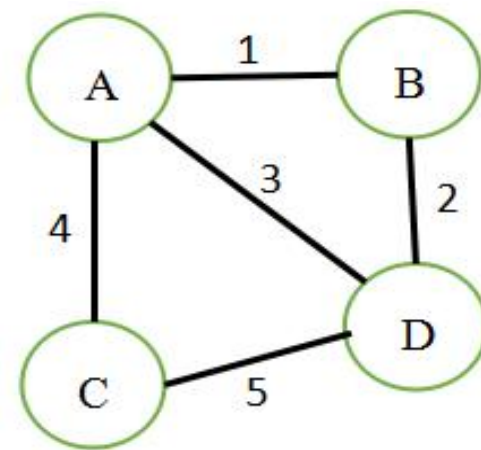
时间复杂度: $O(n)$

第4章 贪心法

- 贪心法：广泛应用于最优化问题求解的算法设计中。
- 对于许多NP难的组合优化问题，目前没有找到有效的算法，可使用贪心法设计近似算法求得满足问题要求的近似解。
- 应用实例：哈夫曼编码、最小生成树、单源点最短路径。
- 贪心法的**基本思路**是在对问题求解时总是做出在当前看来是最好的选择，也就是说贪心法不从整体最优上加以考虑，所作出的仅是在某种意义上的局部最优。
- 人们通常希望找到整体最优解，所以采用**贪心法需要证明**设计的算法确实是整体最优解或求解了它要解决的问题。



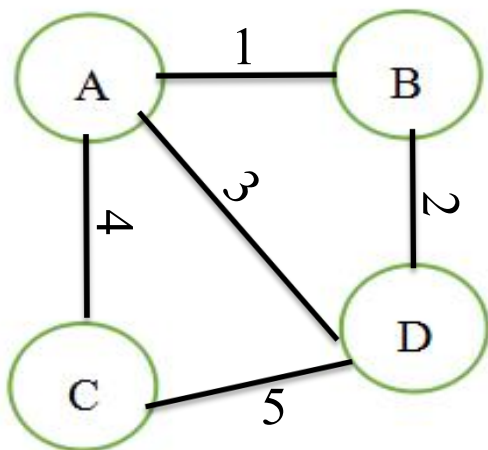
4.1 贪心法的设计思想



□ 最小生成树——克鲁斯卡尔算法

- 将边按照权值从小到大**排序**

$$T = \{ (A, B, 1) (B, D, 2) (A, D, 3) (A, C, 4) (C, D, 5) \}$$

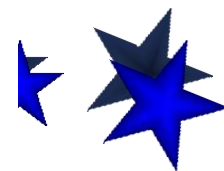


$$X = \{$$

● 候选边加入解集合的条件

- ✓ 当前权值最小
- ✓ 加入图中，不会产生回路

} 通过5次决策，得到
问题的解集合X。



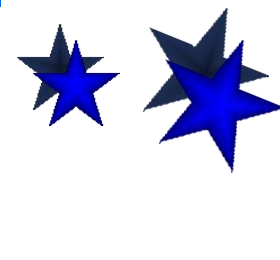
4.1 贪心法的设计思想

□ 贪心法的基本思想

- 贪心法从问题的某一个初始解 $\{ \}$ 出发，采用逐步构造最优解的方法向给定的目标前进，每一步决策产生 n -元组解 $(x_1, x_2, \dots, x_n)$ 的一个分量。

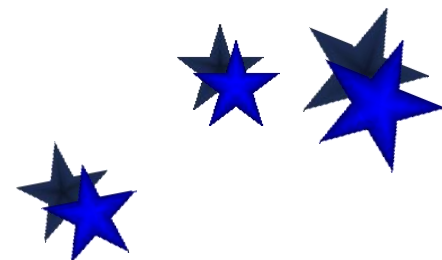
边集T	AB 1	BD 2	AD 3	AC 4	CD 5
序号	0	1	2	3	4
结果集X	1	1	0	1	0

$$X=\{1,1,0,1,0\}$$



4.1 贪心法的设计思想

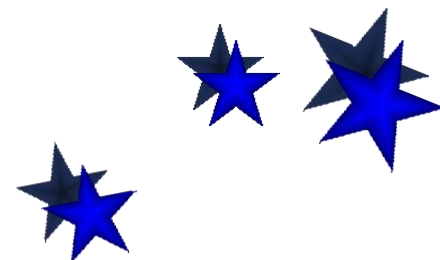
- 每一次贪心选择都将所求问题简化为规模更小的子问题，并通过每次所做的局部最优选择，产生出一个全局最优解。
- 贪心法的每一步作决策，其依据的准则称为最优量度标准（或贪心准则）。也就是说，在选择解分量的过程中，添加新的解分量 x_k 后，形成的部分解 $(x_0, x_1, \dots, x_k)$ 不违反可行解约束条件。



4.1 贪心法的设计思想

- 贪心法求解的问题应具有的性质

- 最优子结构性质：如果一个问题的最优解包含其子问题的最优解，该性质是动态规划算法或贪心法求解的关键性质。
- 所谓贪心选择性质是指所求问题的整体最优解可以通过一系列局部最优的选择，然后再去求解做出这个选择后产生的相应子问题的解，即贪心选择来达到。



第3章 贪心法

```
SolutionType Greedy(SType T[],int n)//贪心法算法框架
{
    Sort(T);                //将数据按照一定的规则排序
    SolutionType X={};       //初始时，解向量X为空集合
    for (int i=0;i<n;i++)    //执行n步决策
    {
        SType xi=Choose(T)  //从候选集T中选择一个当前最好的分量
        if (Feasible(xi))  //判断xi加入到X中是否满足约束条件
            Union(X,xi)    //将xi分量合并到解向量X中
    }
    return X;               //返回生成的最优解——n元组解X
}
```

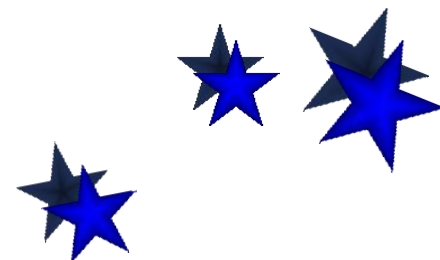


第3章 贪心法

【举例】找零问题：找回的硬币个数最少——贪心策略的应用范围是有限的。

(1) 若硬币的面额为：25分、10分、5分、1分，请给出找回48分钱的方案——贪心法

(2) 若硬币的面额为：25分、10分、1分，则找回30分钱的方案——动态规划



4.1 贪心法的设计思想

■ 贪心法举例1—活动安排问题

- 问题描述：有 n 个活动需要使用公司唯一的活动室，该活动室只能被一个活动占用，且活动开始就不能中断。每个活动提供了开始时间和结束时间。

(1) 会议 i 的开始时间为 b_i ，结束时间为 f_i ，且 $f_i > b_i$ ，因此可以使用区间 $[b_i, f_i)$ 形式化表示会议 i 。

(2) 若某个会议的开始时间晚于另一个会议的结束时间，即会议 $i[b_i, f_i)$ 和会议 $j[b_j, f_j)$ 满足 $b_i \geq f_j$ 或 $b_j \geq f_i$ ，则称这两个会议兼容(相容)。



4.1 贪心法的设计思想

(3) 问题的解：所有会议构成的最大兼容子集。

(4) 数据结构设计：用数组A存放所有的会议， $A[i].b$ ($0 \leq i < n$) 存放会议起始时间， $A[i].f$ 存放会议结束时间。使用一维数组 $flag$ 记录每个会议是否被选中。

(5) 实例

i	1	2	3	4	5	6	7	8	9	10	11
开始	1	3	0	5	3	5	6	8	8	2	12
结束	4	5	6	7	8	9	10	11	12	13	15

要求：设计算法求一种最优安排方案，使得会议室所安排的会议个数最多。

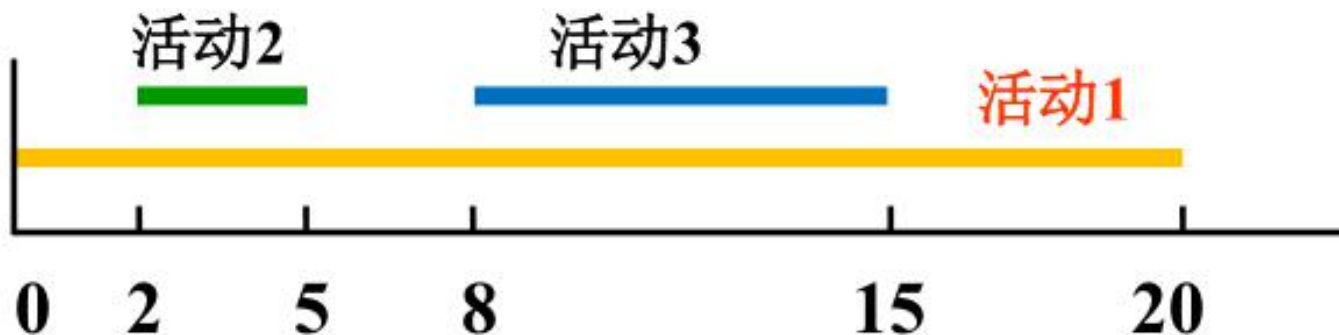
4.1 贪心法的设计思想

- 使用贪心法求解：挑选过程是多步判断，每步依据某种“短视”的策略进行活动选择，选择时注意满足相容性条件（最优子结构性质）。

(1) 贪心想法1：开始时间早的优先排序使 $s_1 \leq s_2 \leq \dots \leq s_n$ ，从前向后挑选。

反例： $S = \{1, 2, 3\}$

$s_1=0, f_1=20, s_2=2, f_2=5, s_3=8, f_3=15$

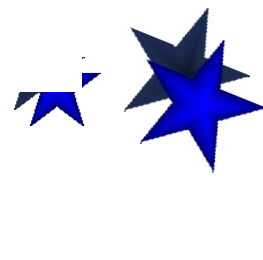
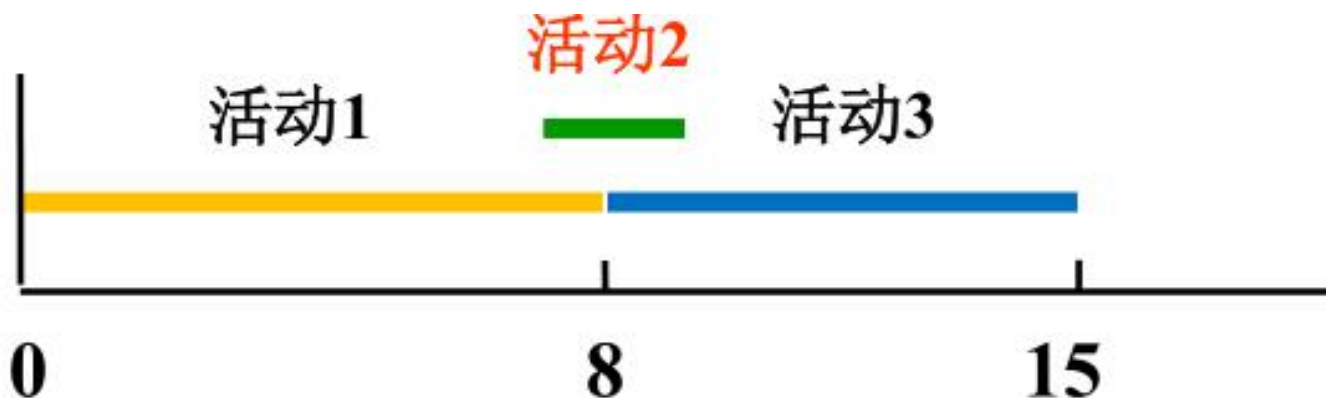


4.1 贪心法的设计思想

(2) 贪心想法2: 占用时间少的优先排序使得 $f_1 - s_1 \leq f_2 - s_2 \leq \dots \leq f_n - s_n$, 从前向后挑选。

反例: $S = \{ 1, 2, 3 \}$

$s_1=0, f_1=8, s_2=7, f_2=9, s_3=8, f_3=15$



4.1 贪心法的设计思想

- 使用策略3求解活动安排问题（按结束时间递增排序）：

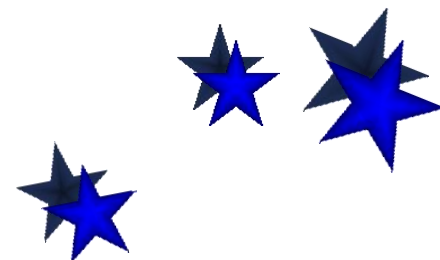
A	1	2	3	4	5	6	7	8	9	10	11
开始时间 b	1	3	0	5	3	5	6	8	8	2	12
结束时间 f	4	5	6	7	8	9	10	11	12	13	15

产生最大兼容活动子集的过程：

<i>flag</i>	1	2	3	4	5	6	7	8	9	10	11
	1	0	0	1	0	0	0	1	0	0	1

$flag = \{1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1\}$,

对应的活动集合 {活动1, 活动4, 活动8, 活动11}



4.1 贪心法的设计思想

- 策略3伪码

算法 Greedy Select

输入：活动集 S , s_i, f_i ,

$i = 1, 2, \dots, n, f_1 \leq \dots \leq f_n$

输出： $A \subseteq S$, 选中的活动子集

1. $n \leftarrow \text{length}[S]$

2. $A \leftarrow \{1\}$

3. $j \leftarrow 1$

已选入的
最后标号

4. for $i \leftarrow 2$ to n do

5. if $s_i \geq f_j$

判相容

6. then $A \leftarrow A \cup \{i\}$

7. $j \leftarrow i$

8. return A

完成时间 $t = \max \{f_k : k \in A\}$

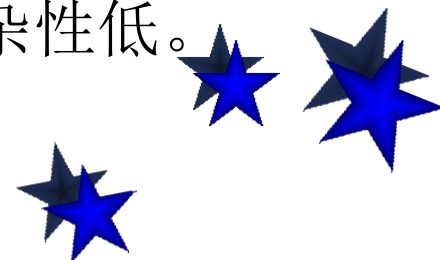


4.1 贪心法的设计思想

□ 贪心法的设计要素：

- (1) 贪心法适用于组合优化问题。
- (2) 求解过程是多步判断过程，最终的判断序列对应于问题的最优解。
- (3) 依据某种“短视的”贪心选择性质判断，性质好坏决定算法的成败。
- (4) **贪心法必须进行正确性证明。**
- (5) 证明贪心法不正确的技巧：举反例。

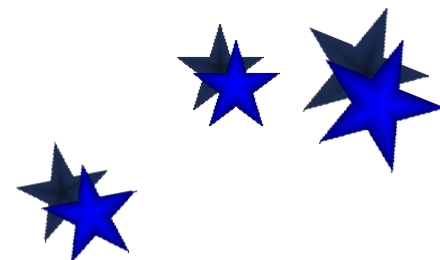
□ 贪心法的优势：算法简单，时间和空间复杂性低。



4.1 贪心法的设计思想

■ 贪心法举例2—可拆包的背包问题

- 问题描述：设有编号为1、2、...、 n 的 n 个物品，它们的重量分别为 w_1 、 w_2 、...、 w_n ，价值分别为 v_1 、 v_2 、...、 v_n ，其中 w_i 、 v_i ($1 \leq i \leq n$) 均为正数。有一个背包可以携带的最大重量不超过 W 。
- 求解目标：在不超过背包负重的前提下，使背包装入的总价值最大（即效益最大化），这里的**每个物品可以取一部分装入背包**。



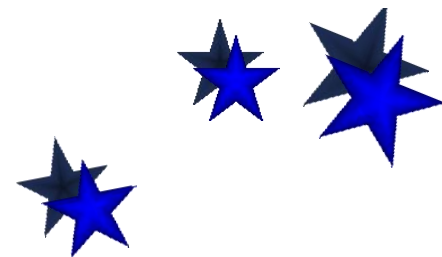
4.1 贪心法的设计思想

- 设 x_i 表示物品 i 装入背包的情况， $0 \leq x_i \leq 1$ 。根据问题的要求，有如下约束条件和目标函数：

$$\sum_{i=1}^n w_i x_i \leq W \quad 0 \leq x_i \leq 1 \quad (1 \leq i \leq n)$$

$$\text{MAX} \left\{ \sum_{i=1}^n v_i x_i \right\}$$

于是问题归结为寻找一个满足上述约束条件，并使目标函数达到最大的解向量 $X = \{x_1, x_2, \dots, x_n\}$ 。



4.1 贪心法的设计思想

实例： $n=3$, $W=50$

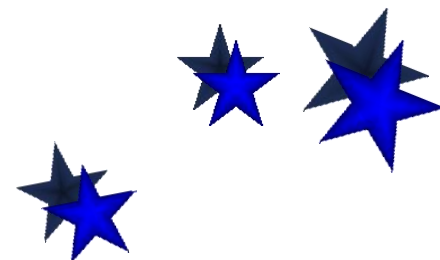
$$(w_1, w_2, w_3)=(20, 30, 10)$$

$$(v_1, v_2, v_3)=(60, 120, 50)$$

对于该背包问题，可以考虑三种看似合理的贪心策略

(1) 选择价值最大的物品，因为这样可以尽可能快地增加背包的总价值。但虽然获得了背包价值的极大增长，但背包的容量可能消耗也很快，从而导致无法获得最优解。

$X=\{1, 1, 0\}$, 重量50, 价值180。



4.1 贪心法的设计思想

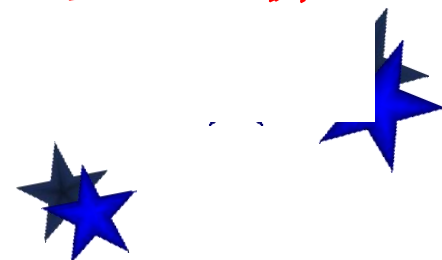
实例： $n=3$ ， $W=50$

$$(w_1, w_2, w_3)=(20, 30, 10)$$

$$(v_1, v_2, v_3)=(60, 120, 50)$$

(2) 选择重量最轻的物体，这样尽可能多的装入物体，增加背包的总价值。虽然背包容量消耗的慢了，但背包的价值却没能保证迅速增长。 **$X=\{1, 2/3, 1\}$ ，重量50，价值190。**

(3) 选择重量价值比（价值 $\div$ 重量）最大的物品，在价值增长与容量消耗之间寻找平衡。 **$X=\{1/2, 1, 1\}$ ，重量50，价值200。**

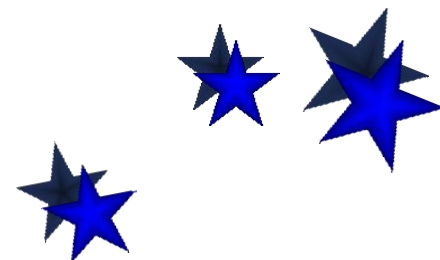


4.1 贪心法的设计思想

选择的贪心策略：选择单位重量价值最大的物品。

每次从物品集合中选择单位重量价值最大的物品，如果其重量小于背包容量，就可以把它装入，并将背包容量减去该物品的重量，然后就面临了一个最优子问题——它同样是背包问题，只不过背包容量减少了，物品集合减少了。

因此背包问题具有最优子结构性质。

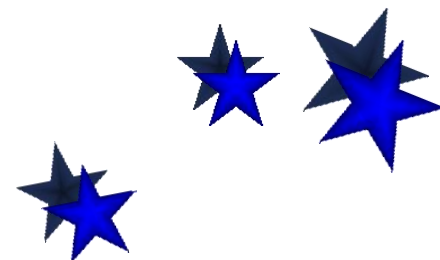


4.1 贪心法的设计思想

□ 使用贪心策略求解背包问题

对于下表一个背包问题， $n=5$ ，设背包容量 $W=100$ ，其

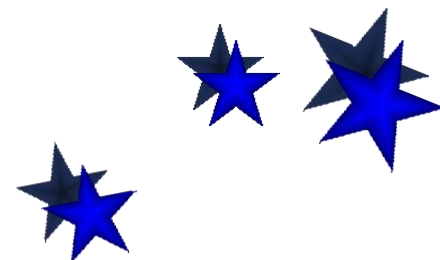
i	1	2	3	4	5
w_i	10	20	30	40	50
v_i	20	30	66	40	60
v_i/w_i	2.0	1.5	2.2	1.0	1.2



4.1 贪心法的设计思想

(1) 将单位价值即 v/w 递减**排序**，物品重新按1~5编号。背包余下装入的重量为 $weight$ ，初始时= W

i	1	2	3	4	5
w_i	10	20	30	40	50
v_i	20	30	66	40	60
v_i/w_i	2.0	1.5	2.2	1.0	1.2

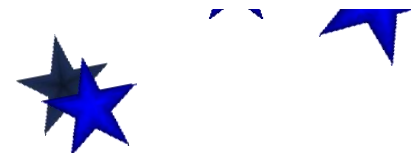


4.1 贪心法的设计思想

i	1	2	3	4	5
w_i	10	20	30	40	50
v_i	20	30	66	40	60
v_i/w_i	2.0	1.5	2.2	1.0	1.2

(2) 从 $i=1$ 开始, $w[1]<weight$ 成立, 表明物品1能够装入, 将其装入到背包中, 置 $x[1]=1$, $weight=weight-w[1]=70$ 。

(3) i 增1即 $i=2$ 。 $w[2]<weight$ 成立, 表明物品2能够装入, 将其装入到背包中, 置 $x[2]=1$, $weight=weight-w[2]=60$ 。



4.1 贪心法的设计思想

i	1	2	3	4	5
w_i	10	20	30	40	50
v_i	20	30	66	40	60
v_i/w_i	2.0	1.5	2.2	1.0	1.2

(4) i 增1即 $i=3$ 。 $w[3]<weight$ 成立，表明物品3能够装入，将其装入到背包中，置 $x[3]=1$ ， $weight=weight-w[3]=50$ 。

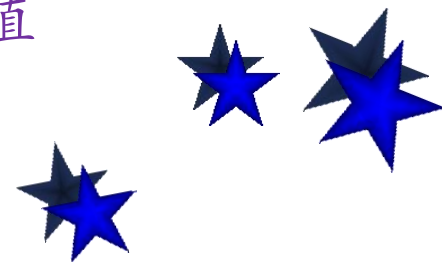
(5) i 增1即 $i=4$ 。 $w[4]<weight$ 不成立，且 $weight>0$ ，表明只能将物品4部分装入，装入比例= $weight/w[4]=50/60=80\%$ ，置 $x[4]=0.8$ 。

■ 算法结束，得到 $X=\{1, 1, 1, 0.8, 0\}$ 。



4.1 贪心法的设计思想

```
int i=1,weight=W;
while (A[i].w<weight) //物品i能够全部装入时循环
{   x[i]=1;           //装入物品i
    weight-=A[i].w;    //减少背包中能装入的余下重量
    V+=A[i].v;         //累计总价值
    i++;              //继续循环
}
if (weight>0)          //当余下重量大于0
{   x[i]=weight/A[i].w; //将物品i的一部分装入
    V+=x[i]*A[i].v;     //累计总价值
}
```



4.1 贪心法的设计思想

【注意】0/1背包问题不适合使用贪心算法求最优解

(1) 由于物品不允许分割，因此，无法保证最终能将背包装满，部分闲置的容量降低了背包的单位重量价值。

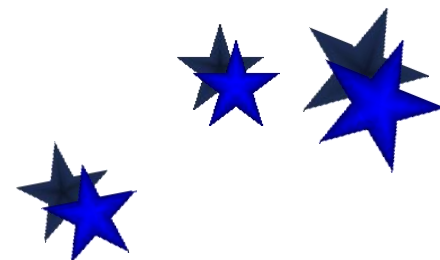
(2) 举例：使用贪婪算法求解0/1背包问题 $W=50$

$$w=\{20,30,10\}$$

$$v=\{60,120,50\}$$

贪心求得的最优解为：重量为40、价值为170

真正的最优解为：重量为50、价值为180.



贪心算法的时间复杂度分析

【算法分析】对于贪心算法包括两个环节：

(1) 排序环节：排序的时间复杂度为 $O(n\log n)$

(2) 贪心选择环节：一重循环，时间复杂度为： $O(n)$

所以贪心算法的时间复杂度为 $O(n\log n)$ 。

